

OSSP Practical Experiments

Arun Tej(2520030472)

Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations

```
#include <stdio.h>

#include <unistd.h>

#include <sys/wait.h>

#include <stdlib.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("\n--- CHILD PROCESS ---\n");
```

```

printf("PID : %d\n", getpid());
printf("PPID : %d\n", getppid());

printf("Child is going to WAIT/SLEEP...\n");
sleep(10);

printf("Child is now RUNNING...\n");

// CPU intensive work
for (long long i = 0; i < 500000000; i++)
{
    // Busy work
}

printf("Child is TERMINATING...\n");
exit(0);
}
else
{
    // Parent process
    printf("\n--- PARENT PROCESS ---\n");
    printf("PID    : %d\n", getpid());
    printf("PPID   : %d\n", getppid());
    printf("Child PID : %d\n", pid);

    printf("Parent is WAITING for child...\n");

    wait(NULL);

    printf("Child has TERMINATED.\n");
    printf("Parent is TERMINATING...\n");
}

```

```
}

return 0;

}
```

Output:

```
aruntej@LAPTOP-DGRT8C5G:~$ nano experiment-3.c
aruntej@LAPTOP-DGRT8C5G:~$ gcc experiment-3.c -o experiment-3
aruntej@LAPTOP-DGRT8C5G:~$ ./experiment-3

--- PARENT PROCESS ---
PID      : 523

PPID     : 305
--- CHILD PROCESS ---
Child PID : 524
Parent is WAITING for child...
PID      : 524
PPID     : 523
Child is going to WAIT/SLEEP...
Child is now RUNNING...
Child is TERMINATING...
Child has TERMINATED.
Parent is TERMINATING...
aruntej@LAPTOP-DGRT8C5G:~$ |
```